

```

In [5]: import pandas as pd
import sqlalchemy
import heapq

# 1. Database Connection
engine = sqlalchemy.create_engine('postgresql://postgres:admin123@localhost:5432/india_aviati

def get_dijkstra_path(start_node, end_node):
    # Load adjacency list from your 'edges' table
    query = "SELECT source_id, target_id, distance_km FROM edges"
    edges_df = pd.read_sql(query, engine)

    # Build the graph dictionary
    graph = {}
    for _, row in edges_df.iterrows():
        u, v, w = row['source_id'], row['target_id'], row['distance_km']
        if u not in graph: graph[u] = []
        if v not in graph: graph[v] = []
        graph[u].append((v, w))
        graph[v].append((u, w)) # Graph is undirected

    # Dijkstra's Algorithm
    queue = [(0, start_node, [])]
    seen = set()
    mins = {start_node: 0}

    while queue:
        (cost, v1, path) = heapq.heappop(queue)
        if v1 not in seen:
            seen.add(v1)
            path = path + [v1]
            if v1 == end_node:
                return (cost, path)

            for v2, weight in graph.get(v1, []):
                if v2 in seen: continue
                prev = mins.get(v2, None)
                next_cost = cost + weight
                if prev is None or next_cost < prev:
                    mins[v2] = next_cost
                    heapq.heappush(queue, (next_cost, v2, path))

    return float("inf"), []

# 2. Interactive Input
print("--- India Aviation Pathfinding (Dijkstra) ---")
start = input("Enter Source Ident (e.g., VOHS for Hyderabad): ").upper()
goal = input("Enter Destination Ident (e.g., VIDP for Delhi): ").upper()

distance, route = get_dijkstra_path(start, goal)

if route:
    print(f"\nShortest Distance: {distance:.2f} km")
    print(f"Path: {' -> '.join(route)}")
else:
    print("\nNo path found between these nodes.")

```

--- India Aviation Pathfinding (Dijkstra) ---

Shortest Distance: 1891.02 km
Path: VOIC -> VOTP -> VEBS -> VECC

```

In [6]: import folium

```

```

def plot_path_on_map(route_idsents, distance):
    # 1. Fetch coordinates for the route from Postgres
    query = f"SELECT ident, name, latitude_deg, longitude_deg FROM airports WHERE ident IN {tuple(route_idsents)}"
    route_data = pd.read_sql(query, engine).set_index('ident').loc[route_idsents]

    # 2. Initialize map centered on India
    m = folium.Map(location=[20.5937, 78.9629], zoom_start=5, tiles="cartodbpositron")

    # 3. Extract coordinates for the line
    points = []
    for ident, row in route_data.iterrows():
        point = (row['latitude_deg'], row['longitude_deg'])
        points.append(point)

        # Add markers for each airport in the path
        folium.Marker(
            location=point,
            popup=f"{row['name']} ({ident})",
            icon=folium.Icon(color='blue', icon='plane')
        ).add_to(m)

    # 4. Draw the Path (The Edges)
    folium.PolyLine(points, color="red", weight=2.5, opacity=0.8,
        tooltip=f"Shortest Path: {distance:.2f} km").add_to(m)

    return m

# Example Usage:
# Assuming 'route' and 'distance' are variables from your previous Dijkstra run
map_view = plot_path_on_map(route, distance)
map_view.save("dijkstra_result.html")
map_view

```

Out[6]:

